

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ**

Звіт з виконання лабораторної роботи №1
з дисципліни «Спеціальні розділи обчислювальної математики»

БАГАТОРОЗРЯДНА АРИФМЕТИКА

Виконала студентка
групи ФІ-43
Репко Єлизавета Сергіївна

Зміст

1	Мета роботи	3
2	Завдання до комп'ютерного практикуму	3
3	Теоретичні відомості	3
4	Результати роботи програми	4
5	Висновки	5
6	Додаток. Лістинг коду програми	7
6.1	Файл BigInt.h	7
6.2	Файл BigInt.cpp	8
6.3	Файл tests.h	13
6.4	Файл tests.cpp	13
6.5	Файл benchmarks.h	14
6.6	Файл benchmarks.cpp	14
6.7	Файл main.cpp	16

1 Мета роботи

Отримання практичних навичок програмної реалізації багаторозрядної арифметики; ознайомлення з прийомами ефективної реалізації критичних по часу ділянок програмного коду та методами оцінки їх ефективності.

2 Завдання до комп'ютерного практикуму

Пункт А. Розробити клас для роботи з багаторозрядними цілими числами. Реалізувати такі операції (включно з бажаними):

- переведення малих констант у формат великого числа;
- додавання та віднімання чисел;
- множення чисел, піднесення чисел до квадрату;
- ділення чисел, знаходження остачі від ділення;
- піднесення числа до багаторозрядного степеня;
- конвертування числа в символьну строку (обов'язкова підтримка шістнадцяткового представлення, а також бажана десяткового та двійкового);
- визначення номеру старшого ненульового біта та бітові зсуви.

Пункт Б. Проконтролювати коректність реалізації алгоритмів шляхом перевірки тотожностей (дистрибутивність, комутативність, багаторазове додавання тощо).

Пункт В. Обчислити середній час виконання реалізованих арифметичних операцій.

3 Теоретичні відомості

У криптографічних застосуваннях найчастіше використовується представлення чисел як беззнакових із фіксованою довжиною. У даній роботі числа реалізовані у вигляді об'єктів класу `BigInt`, що містять масив із 32-х бітних слів фіксованої довжини ($32 \times 32 = 1024$ біти) у форматі little-endian.

Ділення та знаходження остачі реалізовано за алгоритмом «зсувай і віднімай». Піднесення до степеня реалізовано за допомогою схеми Горнера, що опрацьовує двійковий запис показника степеня від старших бітів до молодших, виконуючи послідовні піднесення до квадрату та множення на базу.

4 Результати роботи програми

Нижче наведено символіні зображення тестових чисел та результатів операцій над ними (Пункт А, Пункт Б та Пункт В).

--- Running Tests ---

Test 1 [(a+b)*c == a*c + b*c]: Passed!
Test 2 [100*a == a+a+...+a]: Passed!
Test 3 [a+b == b+a]: Passed!
Test 4 [a*b == b*a]: Passed!
Test 5 [A == B*(A/B) + (A%B)]: Passed!
Test 6 [a^3 == a*a*a]: Passed!
Test 7 [a^2 == a*a]: Passed!

--- Running Benchmarks ---

Operation | Average Time (ns)

Addition	30
Subtract	33
Multiply	101
Square	98
Division	684
Modulo	412
Power	639

A = aabbccdd11223344
B = 1111111111111111
A + B = bbccdde22334455
A - B = 99aabbcc00112233

c < d: 1 (expected 1)
d == d: 1 (expected 1)
c >= d: 0 (expected 0)

testNum = f
bitLength: 4 (expected 4)
testNum << 4: f0 (expected f0)
shiftedHigh >> 4: f (expected f)

100000000 * 200000000 = 20000000000000000 (expected 20000000000000000)
FFFFFFFF * FFFFFFFF = ffffffffe00000001 (expected ffffffffe00000001)

```
100000000 / 3 = 55555555 (expected 55555555)
100000000 % 3 = 1 (expected 1)
A / B = a (expected a)
A % B = 1122326677889a (expected 1122326677889a)
```

```
5 ^ 13 = 1158e460913d (expected 1158e460913d)
```

```
--- Formatting Tests ---
```

```
Hex: ff
```

```
Binary: 11111111 (expected 11111111)
```

```
Decimal: 255 (expected 255)
```

5 Висновки

Під час виконання лабораторної роботи було розроблено клас `BigInt` для багаторозрядної арифметики на мові C++ (модель зберігання — масив із 32-бітних беззнакових цілих чисел). Усі реалізовані оператори пройшли перевірку на математичних тотожностях, довівши коректність роботи алгоритмів.

Аналіз продуктивності повністю підтвердив теоретичну асимптотичну складність реалізованих алгоритмів:

- **Додавання та віднімання (~30–33 нс):** Продемонстрували найшвидший час виконання. Це пояснюється їхньою лінійною асимптотичною складністю $O(n)$, де n — кількість 32-бітних блоків (у нашому випадку $n = 32$). Алгоритм потребує лише одного лінійного проходу по масивах операндів.
- **Множення (~101 нс) та Квадрат (~98 нс):** Реалізований класичний алгоритм «у стовпчик» має квадратичну складність $O(n^2)$, оскільки використовує подвійний вкладений цикл по блоках чисел. Час виконання логічно зріс у порівнянні з додаванням, однак завдяки оптимізації (пропуск нульових старших блоків через `continue`) на тестових даних вдалося мінімізувати кількість ітерацій внутрішнього циклу.
- **Ділення (~684 нс) та остача (~412 нс):** Виявилися найбільш ресурсомісткими базовими операціями. Використаний алгоритм «зсувай і віднімай» має складність $O(n^2)$, проте відрізняється великою константою прихованих витрат: на кожній ітерації циклу `while` виконуються «важкі» виклики методу `bitLength()`, створення нових об'єктів для побітових зсувів на великі відстані та багаторозрядне віднімання. Остача працює швидше за ділення, оскільки не потребує додаткових операцій зсуву та додавання для накопичення частки `q`.

- **Піднесення до степеня (~ 639 нс):** Реалізовано за допомогою схеми Горнера (двійкове піднесення до степеня). Її асимптотична складність становить $O(t \cdot M(n))$, де t — кількість бітів показника степеня, а $M(n)$ — складність операції множення $O(n^2)$. Алгоритм зводить завдання до щонайбільше $2t$ багаторозрядних множень (квадрування та множення на базу), що дозволяє ефективно працювати з великими показниками без експоненційного зростання часу обчислень.

6 Додаток. Лістинг коду програми

Повний початковий код лабораторної роботи та історія комітів доступні у публічному репозиторії на GitHub:

<https://github.com/rrieye/SoROM>

6.1 Файл BigInt.h

```
1 #pragma once
2
3 #include <iostream>
4 #include <array>
5 #include <string>
6 #include <cstdint>
7
8 class BigInt
9 {
10 public:
11     static constexpr size_t WORDS = 32;
12
13     std::array<uint32_t, WORDS> words{};
14
15     BigInt() = default;
16     BigInt(uint32_t value);
17     BigInt(const std::string& hexStr);
18
19     BigInt operator+(const BigInt& other) const;
20     BigInt operator-(const BigInt& other) const;
21
22     BigInt operator*(const BigInt& other) const;
23     BigInt operator*(uint32_t other) const;
24
25     BigInt square() const;
26
27     BigInt operator/(const BigInt& other) const;
28     BigInt operator%(const BigInt& other) const;
29
30     BigInt power(const BigInt& exp) const;
31
32     bool operator<(const BigInt& other) const;
33     bool operator>(const BigInt& other) const;
34     bool operator<=(const BigInt& other) const;
35     bool operator>=(const BigInt& other) const;
36     bool operator==(const BigInt& other) const;
37     bool operator!=(const BigInt& other) const;
38
39     int bitLength() const;
40     BigInt shiftBitsHigh(int shift) const;
41     BigInt shiftBitsLow(int shift) const;
42
43     std::string toHex() const;
44     std::string toBinary() const;
45     std::string toDecimal() const;
46     friend std::ostream& operator<<(std::ostream& os, const BigInt& number);
47 };
```

6.2 Файл BigInt.cpp

```
1 #include "BigInt.h"
2 #include <stdexcept>
3 #include <iomanip>
4 #include <sstream>
5 #include <algorithm>
6
7 using namespace std;
8
9 BigInt::BigInt(uint32_t value)
10 {
11     words[0] = value;
12 }
13
14 BigInt::BigInt(const std::string& hexStr)
15 {
16     int len = hexStr.length();
17     int wordIndex = 0;
18
19     for (int i = len; i > 0; i -= 8)
20     {
21         if (wordIndex >= WORDS) break;
22         int start = std::max(0, i - 8);
23         int count = i - start;
24         std::string chunk = hexStr.substr(start, count);
25         words[wordIndex] = std::stoul(chunk, nullptr, 16);
26         wordIndex++;
27     }
28 }
29
30 BigInt BigInt::operator+(const BigInt& other) const
31 {
32     BigInt result;
33     uint32_t carry = 0;
34     for (size_t i = 0; i < WORDS; ++i)
35     {
36         uint64_t temp = static_cast<uint64_t>(this->words[i]) + other.words[i]
37             + carry;
38         result.words[i] = static_cast<uint32_t>(temp & 0xFFFFFFFF);
39         carry = static_cast<uint32_t>(temp >> 32);
40     }
41     return result;
42 }
43
44 BigInt BigInt::operator-(const BigInt& other) const
45 {
46     BigInt result;
47     uint32_t borrow = 0;
48     for (size_t i = 0; i < WORDS; ++i)
49     {
50         int64_t temp = static_cast<int64_t>(this->words[i]) - other.words[i]
51             - borrow;
52         if (temp >= 0)
53         {
54             result.words[i] = static_cast<uint32_t>(temp);
55             borrow = 0;
56         }
57         else
58         {
59             result.words[i] = static_cast<uint32_t>(temp + 1);
60             borrow = 1;
61         }
62     }
63     return result;
64 }
```



```

57         result.words[i] = static_cast<uint32_t>((1ULL << 32) + temp);
58         borrow = 1;
59     }
60 }
61 if (borrow == 1)
62     throw std::invalid_argument("Error: subtraction result is negative");
63 return result;
64 }
65
66 BigInt BigInt::operator*(uint32_t other) const
67 {
68     BigInt result;
69     uint32_t carry = 0;
70     for (size_t i = 0; i < WORDS; ++i)
71     {
72         uint64_t temp = static_cast<uint64_t>(this->words[i]) * other +
73             carry;
74         result.words[i] = static_cast<uint32_t>(temp & 0xFFFFFFFF);
75         carry = static_cast<uint32_t>(temp >> 32);
76     }
77     return result;
78 }
79
80 BigInt BigInt::square() const
81 {
82     return (*this) * (*this);
83 }
84
85 BigInt BigInt::operator*(const BigInt& other) const
86 {
87     BigInt result;
88     for (size_t i = 0; i < WORDS; ++i)
89     {
90         if (this->words[i] == 0) continue;
91         uint32_t carry = 0;
92         for (size_t j = 0; j < WORDS - i; ++j)
93         {
94             uint64_t temp = static_cast<uint64_t>(result.words[i + j]) +
95                 static_cast<uint64_t>(this->words[i]) * other.
96                     words[j] + carry;
97             result.words[i + j] = static_cast<uint32_t>(temp & 0xFFFFFFFF);
98             carry = static_cast<uint32_t>(temp >> 32);
99         }
100     }
101     return result;
102 }
103
104 BigInt BigInt::operator/(const BigInt& other) const
105 {
106     if (other == BigInt(0)) throw std::invalid_argument("Error: division by
107         zero");
108     BigInt q;
109     BigInt r = *this;
110     int k = other.bitLength();
111     while (r >= other)
112     {
113         int t = r.bitLength();
114         BigInt c = other.shiftBitsHigh(t - k);
115         if (r < c)

```

```

113     {
114         t = t - 1;
115         c = other.shiftBitsHigh(t - k);
116     }
117     r = r - c;
118     BigInt powerOfTwo(1);
119     q = q + powerOfTwo.shiftBitsHigh(t - k);
120 }
121 return q;
122 }
123
124 BigInt BigInt::operator%(const BigInt& other) const
125 {
126     if (other == BigInt(0)) throw std::invalid_argument("Error: division by
127         zero");
128     BigInt r = *this;
129     int k = other.bitLength();
130     while (r >= other)
131     {
132         int t = r.bitLength();
133         BigInt c = other.shiftBitsHigh(t - k);
134         if (r < c)
135         {
136             t = t - 1;
137             c = other.shiftBitsHigh(t - k);
138         }
139         r = r - c;
140     }
141     return r;
142 }
143
144 BigInt BigInt::power(const BigInt& exp) const
145 {
146     BigInt result(1);
147     int bits = exp.bitLength();
148     for (int i = bits - 1; i >= 0; --i)
149     {
150         uint32_t bit = (exp.words[i / 32] >> (i % 32)) & 1;
151         if (bit == 1) result = result * (*this);
152         if (i > 0) result = result.square();
153     }
154     return result;
155 }
156
157 bool BigInt::operator<(const BigInt& other) const
158 {
159     int i = WORDS - 1;
160     while (i >= 0 && this->words[i] == other.words[i]) i--;
161     if (i == -1) return false;
162     return this->words[i] < other.words[i];
163 }
164
165 bool BigInt::operator>(const BigInt& other) const
166 {
167     int i = WORDS - 1;
168     while (i >= 0 && this->words[i] == other.words[i]) i--;
169     if (i == -1) return false;
170     return this->words[i] > other.words[i];
171 }

```

```

172 bool BigInt::operator<=(const BigInt& other) const
173 {
174     return *this < other || *this == other;
175 }
176
177 bool BigInt::operator>=(const BigInt& other) const
178 {
179     return *this > other || *this == other;
180 }
181
182 bool BigInt::operator==(const BigInt& other) const
183 {
184     int i = WORDS - 1;
185     while (i >= 0 && this->words[i] == other.words[i]) i--;
186     return i == -1;
187 }
188
189 bool BigInt::operator!=(const BigInt& other) const
190 {
191     return !(*this == other);
192 }
193
194 int BigInt::bitLength() const
195 {
196     for (int i = WORDS - 1; i >= 0; --i)
197     {
198         if (words[i] != 0)
199         {
200             int length = i * 32;
201             uint32_t temp = words[i];
202             while (temp > 0)
203             {
204                 length++;
205                 temp >>= 1;
206             }
207             return length;
208         }
209     }
210     return 0;
211 }
212
213 BigInt BigInt::shiftBitsHigh(int shift) const
214 {
215     BigInt result;
216     int wordShift = shift / 32;
217     int bitShift = shift % 32;
218     if (wordShift >= WORDS) return result;
219     uint32_t carry = 0;
220     for (size_t i = 0; i < WORDS - wordShift; ++i)
221     {
222         uint32_t current = this->words[i];
223         result.words[i + wordShift] = (bitShift == 0 ? current : (current <<
            bitShift)) | carry;
224         carry = (bitShift == 0 ? 0 : (current >> (32 - bitShift)));
225     }
226     return result;
227 }
228
229 BigInt BigInt::shiftBitsLow(int shift) const
230 {

```

```

231     BigInt result;
232     int wordShift = shift / 32;
233     int bitShift = shift % 32;
234     if (wordShift >= WORDS) return result;
235     uint32_t carry = 0;
236     for (int i = WORDS - 1; i >= wordShift; --i)
237     {
238         uint32_t current = this->words[i];
239         result.words[i - wordShift] = (bitShift == 0 ? current : (current >>
240             bitShift)) | carry;
241         carry = (bitShift == 0 ? 0 : (current << (32 - bitShift)));
242     }
243     return result;
244 }
245 std::string BigInt::toHex() const
246 {
247     std::stringstream ss;
248     ss << std::hex << std::setfill('0');
249     bool leadingZeros = true;
250     for (int i = WORDS - 1; i >= 0; --i)
251     {
252         if (words[i] != 0 || !leadingZeros)
253         {
254             if (leadingZeros)
255             {
256                 ss << words[i];
257                 leadingZeros = false;
258             }
259             else
260             {
261                 ss << std::setw(8) << words[i];
262             }
263         }
264     }
265     if (leadingZeros) return "0";
266     return ss.str();
267 }
268
269 std::ostream& operator<<(std::ostream& os, const BigInt& number)
270 {
271     os << number.toHex();
272     return os;
273 }
274
275 std::string BigInt::toBinary() const
276 {
277     int len = bitLength();
278     if (len == 0) return "0";
279
280     std::string result = "";
281     for (int i = len - 1; i >= 0; --i)
282     {
283         uint32_t bit = (words[i / 32] >> (i % 32)) & 1;
284         result += (bit == 1 ? '1' : '0');
285     }
286     return result;
287 }
288
289 std::string BigInt::toDecimal() const

```

```

290 {
291     if (*this == BigInt(0)) return "0";
292
293     BigInt temp = *this;
294     BigInt ten("A");
295     std::string result = "";
296
297     while (temp > BigInt(0))
298     {
299         BigInt rem = temp % ten;
300         result += std::to_string(rem.words[0]);
301         temp = temp / ten;
302     }
303
304     std::reverse(result.begin(), result.end());
305     return result;
306 }

```

6.3 Файл tests.h

```

1 #pragma once
2
3 void runTests();

```

6.4 Файл tests.cpp

```

1 #include "tests.h"
2 #include "BigInt.h"
3 #include <iostream>
4
5 using namespace std;
6
7 void runTests()
8 {
9     cout << "--- Running Tests ---\n";
10
11     BigInt a("1122334455667788");
12     BigInt b("99AABBCCDDEEFF00");
13     BigInt c("1234567890ABCDEF");
14
15     BigInt left1 = (a + b) * c;
16     BigInt right1 = (a * c) + (b * c);
17     cout << "Test 1 [(a+b)*c == a*c + b*c]: ";
18     if (left1 == right1) cout << "Passed!\n"; else cout << "Failed!\n";
19
20     BigInt sumResult("0");
21     for (int i = 0; i < 100; ++i)
22     {
23         sumResult = sumResult + a;
24     }
25     BigInt mulResult = a * 100;
26     cout << "Test 2 [100*a == a+a...+a]: ";
27     if (sumResult == mulResult) cout << "Passed!\n"; else cout << "Failed!\n";
28     cout << "\n";

```

```

29     cout << "Test 3 [a+b == b+a]: ";
30     if ((a + b) == (b + a)) cout << "Passed!\n"; else cout << "Failed!\n";
31
32     cout << "Test 4 [a*b == b*a]: ";
33     if ((a * b) == (b * a)) cout << "Passed!\n"; else cout << "Failed!\n";
34
35     BigInt divResult = a / c;
36     BigInt modResult = a % c;
37     BigInt restoredA = (c * divResult) + modResult;
38     cout << "Test 5 [A == B*(A/B) + (A%B)]: ";
39     if (a == restoredA) cout << "Passed!\n"; else cout << "Failed!\n";
40
41     BigInt exponent("3");
42     BigInt powerResult = a.power(exponent);
43     BigInt manualPower = a * a * a;
44     cout << "Test 6 [a^3 == a*a*a]: ";
45     if (powerResult == manualPower) cout << "Passed!\n"; else cout << "
        Failed!\n";
46
47     BigInt squareResult = a.square();
48     BigInt manualSquare = a * a;
49     cout << "Test 7 [a^2 == a*a]: ";
50     if (squareResult == manualSquare) cout << "Passed!\n"; else cout << "
        Failed!\n";
51
52     cout << "-----\n\n";
53 }

```

6.5 Файл benchmarks.h

```

1 #pragma once
2
3 void runBenchmarks();

```

6.6 Файл benchmarks.cpp

```

1 #include "benchmarks.h"
2 #include "BigInt.h"
3 #include <iostream>
4 #include <chrono>
5
6 using namespace std;
7
8 void runBenchmarks()
9 {
10     cout << "--- Running Benchmarks ---\n";
11
12     BigInt a("FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF");
13     BigInt b("11111111111111111111111111111111");
14     BigInt exp("10");
15
16     const int ITERATIONS = 10000;
17     volatile uint32_t dummy = 0;
18
19     auto start = std::chrono::steady_clock::now();

```

```

20  for (int i = 0; i < ITERATIONS; ++i)
21  {
22      BigInt res = a + b;
23      dummy = res.words[0];
24  }
25  auto end = std::chrono::steady_clock::now();
26  long long timeAdd = std::chrono::duration_cast<std::chrono::nanoseconds>
    >(end - start).count() / ITERATIONS;
27
28  start = std::chrono::steady_clock::now();
29  for (int i = 0; i < ITERATIONS; ++i)
30  {
31      BigInt res = a - b;
32      dummy = res.words[0];
33  }
34  end = std::chrono::steady_clock::now();
35  long long timeSub = std::chrono::duration_cast<std::chrono::nanoseconds>
    >(end - start).count() / ITERATIONS;
36
37  start = std::chrono::steady_clock::now();
38  for (int i = 0; i < ITERATIONS; ++i)
39  {
40      BigInt res = a * b;
41      dummy = res.words[0];
42  }
43  end = std::chrono::steady_clock::now();
44  long long timeMul = std::chrono::duration_cast<std::chrono::nanoseconds>
    >(end - start).count() / ITERATIONS;
45
46  start = std::chrono::steady_clock::now();
47  for (int i = 0; i < ITERATIONS; ++i)
48  {
49      BigInt res = a.square();
50      dummy = res.words[0];
51  }
52  end = std::chrono::steady_clock::now();
53  long long timeSquare = std::chrono::duration_cast<std::chrono::
    nanoseconds>(end - start).count() / ITERATIONS;
54
55  start = std::chrono::steady_clock::now();
56  for (int i = 0; i < ITERATIONS; ++i)
57  {
58      BigInt res = a / b;
59      dummy = res.words[0];
60  }
61  end = std::chrono::steady_clock::now();
62  long long timeDiv = std::chrono::duration_cast<std::chrono::nanoseconds>
    >(end - start).count() / ITERATIONS;
63
64  start = std::chrono::steady_clock::now();
65  for (int i = 0; i < ITERATIONS; ++i)
66  {
67      BigInt res = a % b;
68      dummy = res.words[0];
69  }
70  end = std::chrono::steady_clock::now();
71  long long timeMod = std::chrono::duration_cast<std::chrono::nanoseconds>
    >(end - start).count() / ITERATIONS;
72
73  start = std::chrono::steady_clock::now();

```

```

74     for (int i = 0; i < ITERATIONS; ++i)
75     {
76         BigInt res = a.power(exp);
77         dummy = res.words[0];
78     }
79     end = std::chrono::steady_clock::now();
80     long long timePow = std::chrono::duration_cast<std::chrono::nanoseconds>
        >(end - start).count() / ITERATIONS;
81
82     cout << "Operation | Average Time (ns)\n";
83     cout << "-----\n";
84     cout << "Addition   | " << timeAdd << "\n";
85     cout << "Subtract   | " << timeSub << "\n";
86     cout << "Multiply   | " << timeMul << "\n";
87     cout << "Square     | " << timeSquare << "\n";
88     cout << "Division   | " << timeDiv << "\n";
89     cout << "Modulo     | " << timeMod << "\n";
90     cout << "Power      | " << timePow << "\n";
91     cout << "-----\n\n";
92 }

```

6.7 Файл main.cpp

```

1  #include <iostream>
2  #include "BigInt.h"
3  #include "tests.h"
4  #include "benchmarks.h"
5
6  using namespace std;
7
8  int main()
9  {
10     runTests();
11     runBenchmarks();
12
13     BigInt a("AABBCCDD11223344");
14     BigInt b("1111111111111111");
15     BigInt c("100");
16     BigInt d("200");
17     BigInt testNum("F");
18
19     cout << "A = " << a << "\n";
20     cout << "B = " << b << "\n";
21     cout << "A + B = " << (a + b) << "\n";
22     cout << "A - B = " << (a - b) << "\n\n";
23
24     cout << "c < d: " << (c < d) << " (expected 1)\n";
25     cout << "d == d: " << (d == d) << " (expected 1)\n";
26     cout << "c >= d: " << (c >= d) << " (expected 0)\n\n";
27
28     cout << "testNum = " << testNum << "\n";
29     cout << "bitLength: " << testNum.bitLength() << " (expected 4)\n";
30
31     BigInt shiftedHigh = testNum.shiftBitsHigh(4);
32     cout << "testNum << 4: " << shiftedHigh << " (expected f0)\n";
33
34     BigInt shiftedLow = shiftedHigh.shiftBitsLow(4);
35     cout << "shiftedHigh >> 4: " << shiftedLow << " (expected f)\n";

```



```

36
37     BigInt mul1("100000000");
38     BigInt mul2("200000000");
39     cout << "\n100000000 * 200000000 = " << (mul1 * mul2) << " (expected
        200000000000000000)\n";
40
41     BigInt mul3("FFFFFFFF");
42     BigInt mul4("FFFFFFFF");
43     cout << "FFFFFFFF * FFFFFFFF = " << (mul3 * mul4) << " (expected
        ffffffffe00000001)\n";
44
45     BigInt div1("100000000");
46     BigInt div2("3");
47
48     cout << "\n100000000 / 3 = " << (div1 / div2) << " (expected 55555555)\n
        ";
49     cout << "100000000 % 3 = " << (div1 % div2) << " (expected 1)\n";
50
51     BigInt div3("AABBCCDD11223344");
52     BigInt div4("1111111111111111");
53     cout << "A / B = " << (div3 / div4) << " (expected a)\n";
54     cout << "A % B = " << (div3 % div4) << " (expected 112326677889a)\n";
55
56     BigInt base("5");
57     BigInt exponent("13");
58
59     cout << "\n5 ^ 13 = " << base.power(exponent) << " (expected 1158
        e460913d)\n";
60
61     BigInt formatTest("FF");
62     cout << "\n--- Formatting Tests ---\n";
63     cout << "Hex: " << formatTest.toHex() << "\n";
64     cout << "Binary: " << formatTest.toBinary() << " (expected 11111111)\n";
65     cout << "Decimal: " << formatTest.toDecimal() << " (expected 255)\n";
66
67     return 0;
68 }

```